

Golang Developer Assessment 2

Networked Multi-Node Blockchain

Turn a single-node chain into a small peer-to-peer network that gossips, syncs, and resolves forks.

Assessment at a glance	
Document	Developer Requirements Specification
Round	Assessment 2, builds on the round 1 toy blockchain
Role and level	Software Engineering Intern (Backend, Go)
Difficulty	Hard, a clear step up from round 1
Primary language	Go 1.22 or newer
Suggested timebox	8 to 10 working days, worked solo
Delivery format	Multi-node application plus a written report
Version	1.0
Date	July 2026

1 Overview and Purpose

In round 1 you built a single-process toy blockchain. In this assessment you turn that program into a small network of independent nodes that talk to each other. Nodes broadcast transactions and blocks to their peers, a fresh node can join and sync the whole chain, and the network agrees on a single history even when two miners produce competing blocks. Everything still runs locally, now as several processes on different ports.

Round 1 tested data structures, hashing, and validation inside one process. This round tests what a single process cannot reveal: networking, concurrency under real contention, distributed agreement, and, just as important, **whether you can extend and refactor your own earlier code** rather than start from a blank page. The research report again carries real weight.

HOW TO APPROACH THIS

Start from your round 1 submission. Refactor it into clean packages first, then layer the network on top. Get two nodes talking before you chase three-way forks. A correct, race-free two-node network beats a feature-rich one that deadlocks or loses blocks.

2 What We Are Evaluating

In rough order of importance, this exercise is designed to reveal:

1. **Networking and protocol design:** a clear, simple way for nodes to exchange transactions, blocks, and chain state.
2. **Concurrency:** safe shared state across mining, gossip, and request handling, proven with the race detector.
3. **Distributed agreement:** correct fork resolution and reorganisation so the network converges on one chain.
4. **Extending existing code:** refactoring your round 1 chain into reusable pieces without breaking what worked.
5. **Research and communication:** experiments that show your network converging, and honest discussion of the tradeoffs.

3 Project Brief

Run several instances of your node on one machine, each on its own port and each aware of one or more peers. When a node receives a new transaction it verifies the signature, adds it to its pool, and forwards it to its peers. When a node mines a block it broadcasts it, and peers validate and append it. A node starting with an empty chain must be able to sync the full history from a peer. When two valid chains compete, every node must end up on the same one by following an agreed rule.

Alongside the code, you will write a short research report (see [Section 7](#)).

4 Scope

4.1 In scope

- Multiple independent node processes, each with an HTTP interface and a peer list.
- Ed25519-signed transactions, verified on every node.
- Gossip of transactions and blocks between peers, with de-duplication.
- Chain synchronisation for a newly started or lagging node.
- Fork resolution and chain reorganisation on a shared rule.
- Race-free shared state, unit tests, and a research report.

4.2 Out of scope

- Third-party peer-to-peer or blockchain frameworks such as libp2p. The point is to build the protocol yourself.
- Cross-machine networking, NAT traversal, or public deployment. Everything runs on localhost.
- Smart contracts, a virtual machine, or any EVM or Solidity work.
- A polished web frontend or graphical interface. Simple HTTP endpoints and logs are enough.
- Cryptographic novelty. Use the standard ed25519 and SHA-256 primitives.

5 Functional Requirements

Priority key: **MUST** required **SHOULD** strongly expected **COULD** nice to have.

FR-1 Node as a networked service MUST

Each node runs as an independent process exposing an HTTP API. Its listen address and an initial peer list are set through flags or a config file. Several nodes must run side by side on one machine using different ports.

FR-2 Signed transactions MUST

Transactions are signed with an ed25519 private key and carry the sender's public key and signature. Every node verifies the signature before accepting a transaction and rejects any that fail. The sender's address is derived from the public key.

FR-3 Transaction gossip MUST

A transaction received from a client or a peer is validated, added to the pending pool if it is new, and forwarded to the node's peers. Nodes must de-duplicate so the same transaction does not loop endlessly around the network.

FR-4 Block gossip MUST

A newly mined block is broadcast to peers. A receiving node validates it fully (proof of work, previous-hash link, signatures, and the resulting ledger) and appends it if it extends the current chain, or treats it as a competing block otherwise.

FR-5 Chain synchronisation MUST

A node that starts with only the genesis block can request a peer's height and download the missing blocks, validating each one, until it matches the peer. The same path lets a node that has fallen behind catch up.

FR-6 Fork resolution and reorganisation MUST

When a node learns of a competing valid chain that is longer, or that carries more total work, it switches to that chain. The switch must rebuild ledger balances for the new chain and return transactions from orphaned blocks to the pending pool if they are still valid.

FR-7 Concurrency safety MUST

The chain, the pending pool, and the peer set are read and written by several goroutines at once: request handlers, the miner, and gossip. Access must be free of data races, and the suite must pass `go test -race ./...`

FR-8 Node introspection API SHOULD

Expose read endpoints for at least: current height and head hash, the list of known peers, the pending pool size, account balances, and a dump of the chain. These make the network observable and testable.

FR-9 Mempool consistency SHOULD

Remove transactions from the pending pool once they appear in an accepted block, and prevent the same funds being spent twice from the pool.

FR-10 Peer health COULD

Exchange known peers on connection and drop peers that stay unreachable, so a small network can form from a single seed address.

FR-11 Persistence across restarts COULD

Reload the chain and the node's key from disk on restart, so a node keeps its identity and history between runs.

6 Non-Functional Requirements

- **Race-free.** `go test -race ./...` must pass. Treat a race as a failing test, not a warning.
- **Runnable as a local cluster.** Provide a script, Make target, or compose file that starts at least three nodes and points them at each other.
- **Observable.** Log peer connections, gossip, block acceptance, and reorganisations clearly enough to follow what the network is doing.
- **Idiomatic and tested.** `gofmt` and `vet` clean, with unit tests for signing, gossip de-duplication, sync, and fork resolution.
- **Documented.** A README covering how to start a cluster, the API, the main design decisions, and a short note on what changed from round 1.

7 Research Component

Write a short report of roughly **3 to 5 pages** (Markdown or PDF), driven by experiments you run against your own network. We care about clear reasoning and honest observation far more than length or polish.

7.1 Required investigations

6. **Convergence after a fork.** Start two nodes, briefly stop them from talking, mine a block on each so the chains diverge, then reconnect them. Show that both nodes end on the same chain, and report which block was orphaned and what happened to its transactions.

7. **Gossip cost.** Measure how many messages travel the network when one transaction is broadcast across three or more nodes, and explain how your de-duplication stops that number from exploding.
8. **Design write-up.** Explain your wire format and endpoints, how you keep shared state race-free (locks versus channels, and why), and exactly how a reorganisation rebuilds the ledger.

7.2 Discussion questions

- The longest-valid-chain rule lets independent nodes agree without a coordinator. Why does it work only probabilistically, and what is a 51 percent attack?
- What does finality mean, and why does a small proof-of-work chain never offer hard finality? How do real networks reduce the risk, for example by waiting for confirmations?
- Your nodes trust each other very little yet still cooperate. Explain what signatures prevent, then give one thing a malicious peer could still try and how your node defends against it.

Cite any sources you rely on. We are testing your ability to research and synthesise, not to memorise.

8 Technical Constraints and Guidelines

- Go 1.22 or newer. Use crypto/ed25519, crypto/sha256, net/http, and encoding/json from the standard library. A small router library is allowed but not required.
- No third-party peer-to-peer or blockchain frameworks. Build the gossip and sync protocol yourself.
- Build on your round 1 code. Refactor freely, but keep the hashing and validation ideas you already proved.
- All nodes run locally on separate ports. No external hosting is needed.
- go test -race ./... must pass. Commit your work in small, meaningful steps.
- AI assistants are permitted, but be ready to explain every line and to walk through a live reorganisation.

9 Acceptance Criteria

The submission is accepted when the following scenarios hold. They are written in Gherkin style and map directly to the functional requirements.

Scenario: A transaction with an invalid signature is rejected (FR-2)

Given a node running with a valid chain
When it receives a transaction whose signature does not match its public key
Then the node rejects the transaction
And the transaction is not added to the pending pool

Scenario: A transaction propagates to a peer (FR-3)

Given two connected nodes A and B
When a valid transaction is submitted to node A
Then the transaction appears in node B's pending pool
And it is not forwarded back to node A a second time

Scenario: A mined block is accepted by a peer (FR-4)

Given two connected nodes at the same height
When node A mines and broadcasts a new block
Then node B validates the block
And node B's height increases by one

Scenario: A new node syncs the chain from a peer (FR-5)

Given an existing node with a chain of several blocks
And a new node that has only the genesis block
When the new node syncs from the existing peer
Then the new node's chain matches the existing peer's chain

Scenario: The network converges after a fork (FR-6)

Given two nodes that each mined a different block at the same height
When the two nodes exchange their chains
Then both nodes adopt the longer valid chain
And any orphaned transactions return to the pending pool

Scenario: Shared state is free of data races (FR-7)

Given a node that is mining while gossiping with peers
When the test suite runs with the race detector enabled
Then no data race is reported

10 Deliverables

9. **Source code** in a Git repository that builds and runs, based on your round 1 project.
10. **A cluster launcher** (script, Make target, or compose file) that starts at least three connected nodes.
11. **README** with cluster start-up, the API, design decisions, and what changed from round 1.
12. **Tests** runnable via `go test -race ./...`
13. **Research report** in Markdown or PDF covering Section 7.

11 Evaluation Rubric

Your submission is scored across the following areas.

Area	Weight	What good looks like
Networking, gossip, and sync	25%	Nodes reliably exchange transactions, blocks, and chain state.
Consensus: fork resolution and reorg	20%	The network converges on one chain and rebuilds state correctly.

Area	Weight	What good looks like
Concurrency safety	15%	The race detector passes under concurrent mining and gossip.
Signed transactions and validation	10%	Signatures are enforced everywhere data enters a node.
Code quality, structure, and refactor	15%	Clean packages; round 1 code extended, not bolted on.
Research report	15%	Real experiments, clear reasoning, honest limits.

12 Suggested Milestones

A guide, not a mandate. Organise the two weeks however suits you.

Phase	Focus
Phase 1	Refactor the round 1 chain into reusable packages. Add ed25519 keys and signed transactions.
Phase 2	Wrap the node as an HTTP service with config for its address and peers, plus the introspection endpoints.
Phase 3	Implement transaction and block gossip with de-duplication, validating everything on receipt.
Phase 4	Add chain sync for new nodes, then fork resolution and reorganisation. Make shared state race-free.
Phase 5	Build the local cluster, write tests including the race detector, run the experiments, and write the report.

13 Stretch Goals

Attempt these only after the core network is solid. Any one of them is a strong signal, and none are required.

- **Heaviest chain.** Choose the chain with the most cumulative proof-of-work rather than the most blocks.
- **Difficulty retargeting.** Adjust difficulty across the network to hold block times roughly steady.
- **Peer discovery.** Let nodes share peer lists so a network forms from a single seed address.
- **Merkle proofs.** Add a Merkle root per block and serve inclusion proofs for a single transaction.
- **Compose cluster.** Ship a docker-compose file that brings up the whole network with one command.

14 Submission

Share a link to a private repository, or send a zipped archive, containing all deliverables. Make sure a clean checkout builds and a fresh cluster starts using only the documented commands. Be ready for a short call

where we start a few nodes together, submit a transaction, watch it propagate, and force a fork so you can walk us through how your network recovers.